

React Component Project

Restaurant Web Application — Weekly Practical Assignment

Project Goal

Build one complete static restaurant website by developing separate React components and combining them into a single application.

This Week's Focus

React project architecture, component responsibility, component styling, folder structure, responsive layout, and integration.

Not covered yet

API calls, advanced state management, backend integration, and complex JavaScript functionality. These will come in the next phase.

Learning Outcome

- Understand how a React page is divided into independent components.
- Create a component with its own responsibility and styling.
- Follow consistent naming and folder conventions.
- Work on an assigned component without breaking other components.
- Combine multiple independently developed components into one application.

Different components → One complete application

1. Weekly Execution Plan

The week is divided into small milestones so that students first understand their component, then build it, and finally experience the integration process.

Day	Focus	Expected Output
Monday	Project introduction + component assignment	Understand the complete UI, identify component boundaries, receive instructions
Tuesday	Component development	Initial JSX structure, assets, and basic styling completed.
Wednesday	UI refinement + responsive layout	Component visually matches the reference and works on smaller screens
Thursday	Component review + fixes	Clean structure, naming, spacing, CSS, and final component ready for integration
Friday / Weekend	Integration + final application	All components combined into one React application and tested together.

Rules for the week

- Build only the component assigned to you unless the instructor asks you to modify another area.
- Keep component-specific styles with the component wherever practical.
- Use meaningful names for components, folders, images, and CSS files.
- Do not replace the whole project with generated code. The purpose is to understand the architecture by building it.
- Ask before changing shared/global styles because one global change can affect everyone.

2. Component Assignment — Complete UI Reference

Use the visual specification below as the reference for what each student should build. The goal is not pixel-perfect copying; the goal is to understand the layout, content, alignment, spacing, and component responsibility.

3. Component Assignment Detailed UI Breakdown

Below is the complete restaurant website divided into individual components. Each student will work on one component. Follow the reference design, required elements, and alignment guide for your assigned section.

"Different components,
One amazing application"



Complete Page Preview

1. Navbar Component

- Logo (left)
- Navigation links (center)
- CTA button (right)
- Sticky on scroll (optional)
- Responsive (hamburger for mobile)

2. Hero Component

- Main heading
- Short description
- CTA button
- Background image or solid color
- Left content, right image alignment (stack on mobile)

3. Categories Component

- Section title + subtitle
- Category icons with labels
- Equal spacing and alignment
- Hover effect (optional)
- Horizontal scroll or wrap on mobile

4. Popular Items Component

- Section title + subtitle
- Food cards with image, name, price, rating
- Add to Cart button
- Grid layout (3 cards, responsive)

5. Offers Component

- Offer title
- Short description
- CTA button
- Offer image
- Attractive background (image or gradient)

6. About Component

- Image (left)
- Title + description (right)
- Stats with icons
- Clean alignment and spacing
- Responsive (stack on mobile)

7. Contact Component

- Section title + subtitle
- Contact information (address, phone, email)
- Contact form (name, email, message)
- Proper alignment (info left, form right)
- Responsive layout

8. Footer Component

- Logo + short tagline
- Social media icons
- Quick links
- Newsletter subscription
- Copyright text
- Clean 3-column alignment (responsive)

3. Component Requirements

1. Navbar

- | | | | |
|---|--|--|--|
| <ul style="list-style-type: none">• Logo / brand on the left | <ul style="list-style-type: none">• Navigation links in the center | <ul style="list-style-type: none">• Primary CTA on the right | <ul style="list-style-type: none">• Keep spacing and vertical alignment consistent |
| <ul style="list-style-type: none">• Mobile layout should provide a reasonable compact arrangement | | | |

2. Hero

- | | | | |
|--|--|---|---|
| <ul style="list-style-type: none">• Main heading and short description | <ul style="list-style-type: none">• Primary CTA button | <ul style="list-style-type: none">• Food image or visual on the right | <ul style="list-style-type: none">• Two-column desktop layout |
| <ul style="list-style-type: none">• Stack content on smaller screens | | | |

3. Categories

- | | | | |
|---|--|---|---|
| <ul style="list-style-type: none">• Section title and subtitle | <ul style="list-style-type: none">• Category icon/image with label | <ul style="list-style-type: none">• Equal card sizing and spacing | <ul style="list-style-type: none">• Consistent horizontal alignment |
| <ul style="list-style-type: none">• Allow wrapping or horizontal scrolling on small screens | | | |

4. Popular Items

- | | | | |
|--|---|---|--|
| <ul style="list-style-type: none">• Section title and subtitle | <ul style="list-style-type: none">• Food image, name, price, rating | <ul style="list-style-type: none">• Add-to-cart CTA | <ul style="list-style-type: none">• Consistent card dimensions |
| <ul style="list-style-type: none">• Responsive grid | | | |

5. Offers

- | | | | |
|---|--|--|--|
| <ul style="list-style-type: none">• Offer title | <ul style="list-style-type: none">• Short promotional text | <ul style="list-style-type: none">• CTA button | <ul style="list-style-type: none">• Large promotional image/background |
| <ul style="list-style-type: none">• Keep content readable over the visual | | | |

6. About

- | | | | |
|---|--|---|--|
| <ul style="list-style-type: none">• Image on one side | <ul style="list-style-type: none">• Title and description on the other | <ul style="list-style-type: none">• Small statistics / highlights | <ul style="list-style-type: none">• Balanced spacing |
| <ul style="list-style-type: none">• Stack into one column on mobile | | | |

7. Contact

- | | | | |
|------------------------------|---------------------------------------|----------------|-----------------------------|
| • Section title and subtitle | • Address / phone / email information | • Contact form | • Two-column desktop layout |
| • Responsive stacking | | | |

8. Footer

- | | | | |
|--------------------------|----------------------------------|-------------------|----------------|
| • Logo and short tagline | • Quick links | • Newsletter area | • Social icons |
| • Copyright text | • Responsive multi-column layout | | |

4. Expected React Architecture

Each component should be treated as an independent unit. The exact structure may be adjusted by the instructor, but students should understand this pattern:

```
src/  
├── components/  
│   ├── Navbar/  
│   │   ├── Navbar.jsx  
│   │   └── Navbar.module.css  
│   ├── Hero/  
│   │   ├── Hero.jsx  
│   │   └── Hero.module.css  
│   ├── Categories/  
│   ├── PopularItems/  
│   ├── Offers/  
│   ├── About/  
│   ├── Contact/  
│   └── Footer/  
├── assets/  
├── App.jsx  
└── main.jsx
```

Component-level checklist

- Component file uses a clear PascalCase name.
- CSS file is kept with the component when using component-scoped styling.
- Images and other reusable assets are placed in the agreed assets location.
- The component does not depend on hardcoded global selectors that can unexpectedly affect other components.
- The component is exported and can be imported into App.jsx.

Global styles

- Use global CSS only for genuinely global concerns such as body margin, font family, page background, reusable typography rules, or shared utility styles.
- Avoid putting every component's CSS into one global stylesheet.

5. Integration Day — Combine Everything

The final exercise is where the individual work becomes a real React application. Students should see that component architecture makes a large page easier to build, review, and maintain.

App.jsx	Navbar
App.jsx	Hero
App.jsx	Categories
App.jsx	PopularItems
App.jsx	Offers
App.jsx	About
App.jsx	Contact
App.jsx	Footer

Integration checklist

- All components import without errors.
- App.jsx composes the page in the correct order.
- No missing images or broken asset paths.
- No accidental CSS conflicts between components.
- Spacing between sections looks consistent.
- The page works at desktop and mobile widths.
- The application runs successfully with the normal Vite development command.

Expected result

One complete restaurant application built from independently developed React components.

6. Definition of Done

- ☐ My assigned component has been created from scratch.
- ☐ The component has a clear responsibility.
- ☐ The JSX structure is clean and readable.
- ☐ The styling is organized and does not unnecessarily leak into other components.
- ☐ The component matches the provided reference layout.
- ☐ The component is reasonably responsive.
- ☐ Images/assets are correctly connected.
- ☐ The component is exported and integrated into App.jsx.
- ☐ The complete application works after integration.
- ☐ I can explain what my component does and where it belongs in the project architecture.

7. What Comes Next

After this static React project is completed, the next phase will introduce JavaScript concepts and use them to make the application dynamic.

Current Week	Next Phase
Components	Variables and expressions
Static UI	Dynamic UI
Hardcoded content	Arrays and objects
Independent components	Props and data flow
Static page	Events and interaction
No API	Fetch / API calls

Main objective: Learn to build React applications by understanding the architecture first, then adding JavaScript behavior.